TRI9T AI

AI Engineering Internship Approach Document

OCR-Based Document Extraction, Versioning and QA Test-Case Generation

Project	CardioTrack CT-200 Home Blood Pressure Monitor
Language	Python
Backend	FastAPI
Persistence	SQLite + SQLAlchemy; JSON/document storage for generated output
Validation	Pydantic + pytest

Working draft — update the marked items with actual findings and test results before submission.

1. Project Overview

The objective is to build a backend API that converts the CT-200 medical-device manual into a structured, searchable and versioned document tree. The system must preserve document hierarchy, identify changes between manual versions, maintain traceability for selected sections, and generate QA test-case ideas using a Large Language Model (LLM).

The main challenge is ensuring that the extracted structure remains accurate when a PDF contains inconsistent formatting, repeated headings, nested sections, tables or other structural variations. The approach prioritizes correctness, traceability and explicit error handling over a clean-looking but potentially incorrect hierarchy.

2. Technology Selection and Rationale

Technology	Purpose and justification
Python	Main language for PDF processing, API development and AI integration.
PyMuPDF	Extracts text and layout information from PDFs.
OCR engine (if required)	Handles scanned pages where selectable text is unavailable or inadequate.
FastAPI	Provides endpoints for browsing, searching, version comparison, selections and QA retrieval.
Pydantic	Validates API requests and structured LLM responses.
SQLAlchemy + SQLite	Persist document versions, nodes, parent-child relationships and selections.
JSON or document store	Stores generated test-case output and its source information.
pytest	Tests parser behavior, version matching, APIs and edge cases.
Git + GitHub	Maintains incremental commits and makes development reviewable.

3. PDF Extraction and Hierarchy Reconstruction

3.1 Inspection before implementation

Before developing the parser, I will inspect both supplied manual versions and compare their structure. I will examine page layouts, heading numbering, font sizes, indentation, paragraph boundaries, lists, tables, figures and captions. I will determine whether each page contains selectable text or requires OCR.

3.2 Extraction strategy

- Process pages in document order and extract text blocks, spans, font information and bounding-box coordinates.
- Use OCR on pages or regions where direct extraction is insufficient.
- Identify candidate headings using numbering patterns, typography, indentation and contextual evidence.
- Reconstruct paragraphs, lists, tables and figure captions while preserving reading order.
- Build a hierarchical tree using heading levels and section boundaries, then validate it before storage.
- Treat numbering and formatting as evidence, not infallible rules; a numbered sentence should not automatically become a heading.

3.3 Hierarchy and parent-child relationships

Each node will have a unique ID, heading, heading level, body text, parent ID, ordered child nodes, document version and content hash. A stack-based hierarchy builder will maintain the current section path. Repeated headings will receive distinct IDs and will not be merged solely because their titles match. If a parent cannot be established confidently, the parser will flag the ambiguity for inspection.

3.4 Structural inconsistencies to investigate

- Duplicate headings in different sections.
- Missing, inconsistent or skipped section numbers.
- Headings whose formatting differs from the expected pattern.
- Wrapped headings and paragraphs spanning page boundaries.
- Lists and tables whose reading order may be disrupted by layout extraction.
- Figures and captions separated by page layout.
- Scanned pages or OCR output containing recognition errors.

Observed irregularities: Replace this placeholder with actual page numbers and examples found during manual inspection.

4. Data Model and Persistence

Document

Stores a stable document ID, version number, source filename, ingestion timestamp and processing status.

DocumentNode

Stores node ID, version ID, heading, level, body text, parent ID, display order, content hash and node type.

Selection

Stores a selection ID, name, creation timestamp and exact version-specific node IDs.

Generation

Stores generation ID, selection ID, model/provider metadata, source node references, source hashes, generated test cases, validation status and timestamp.

This design keeps original versions intact and makes it possible to identify the exact content used to generate each result. Generated output will be stored separately from the document tree so a later ingestion cannot overwrite historical results.

5. Document Versioning and Node Matching

5.1 Content hashing

I will normalize extracted text consistently before computing a cryptographic hash such as SHA-256. Normalization will remove irrelevant formatting differences without changing meaningful content. Each node will retain its own hash.

5.2 Matching strategy

- Match nodes using normalized section path and heading.
- Use parent relationships, section numbering and node type as supporting evidence.
- Use normalized content hashes to identify unchanged content.
- Use similarity-based title and text matching to identify likely renamed or relocated sections.
- Mark unresolved or ambiguous matches for review.

Matching should be one-to-one wherever possible. Identical normalized content will be classified as unchanged; matched nodes with different content as modified; unmatched nodes as added or removed. Both versions will be preserved, with explicit links between corresponding logical nodes.

5.3 Limitations

Hash equality identifies identical normalized text but cannot detect semantic equivalence when wording changes. Similarity matching may confuse repeated headings or substantially rewritten sections. Ambiguous matches will be reported instead of treated as certain.

6. Browse and Selection APIs

The FastAPI backend will provide endpoints to list top-level sections by version; retrieve a node with full text, children and content hash; search headings and body text; compare nodes across versions; create and retrieve named, version-pinned selections; generate QA test-case ideas; and retrieve generated cases with their staleness status.

Selections will reference exact node-version pairs. A later ingestion will not silently redirect an existing selection to new content. Pydantic will validate requests and response models, and invalid IDs, missing versions or malformed requests will return clear error responses.

7. LLM-Powered QA Test-Case Generation

The generation endpoint will reconstruct selected text from the stored version and provide it to an LLM with a structured prompt. The prompt will request three to five concrete QA test-case ideas, each containing a title/objective, preconditions, test steps, expected results and source section/node reference.

The prompt will instruct the model not to invent requirements absent from the manual. Generated cases are suggestions requiring review, not verified medical-device test specifications.

7.1 Output validation and error handling

A Pydantic schema will validate the expected response. The system will handle malformed JSON, missing fields, invalid types, incomplete responses, timeouts and provider errors. A limited retry may use a correction prompt. If validation still fails, the generation will be marked failed and invalid output will not be stored as a successful result.

Each generation will be linked to its selection, source node IDs, source hashes and source version. Identical requests may reuse a valid prior generation under a documented policy; explicit regeneration can create a new record.

8. Staleness and Impact Detection

At retrieval time, stored source hashes will be compared with the corresponding content in the current version. The system will also use version-matching results to identify renamed, removed or ambiguous source sections. The API will report whether output is current, potentially stale or unverifiable.

A changed pressure threshold or safety requirement deserves more attention than a formatting-only change. The system will preserve raw hashes and, where practical, compare normalized text to help explain differences. Semantic comparison cannot guarantee that every clinically meaningful change is detected, so uncertain changes will be exposed for review. Original test cases will not be overwritten or automatically regenerated.

9. Testing and Validation

Automated tests and manual inspection will be used to evaluate extraction and versioning. At least the following unit tests will be included:

- **Duplicate-heading test:** Two sections with the same heading retain distinct node IDs and correct parent relationships.
- **Hierarchy test:** Nested headings receive correct parent-child relationships, including irregular numbering.
- **Version-change test:** An unchanged section is recognized as unchanged, while modified body text is flagged.

Additional tests will cover added and removed sections, malformed LLM responses, invalid API requests, version-pinned selections and staleness retrieval. Extracted text and hierarchy will be compared against the original PDF; suspicious pages will be inspected manually, and each generated test case will be checked for source traceability.

Validation results: Record the actual test commands, results and remaining failures after implementation.

10. Implementation Challenges and Improvements

The initial implementation will be evaluated against the actual document rather than assumed correct after the first extraction. During development, I will record specific failures, the evidence used to identify them and the changes made to resolve them.

Initial implementation failure: Describe an actual issue observed during implementation.

How it was identified: Record the relevant unit test, manual page comparison, validation error or incorrect hierarchy.

Improvement made: Describe the code or logic change and the result after retesting.

Potential failure areas to investigate include heading detection based only on numbering, incorrect parent assignment, page-boundary paragraph splitting, unstable matching of repeated headings and incomplete LLM responses. These are hypotheses to test, not claims that every issue occurs in the supplied manual.

11. Decision Log

11.1 What is most likely to silently produce incorrect results, and how will I catch it?

Incorrect hierarchy reconstruction is a major risk because a section may be extracted correctly but attached to the wrong parent. I will compare the generated tree against the PDF and test parent-child relationships explicitly. Ambiguous cases will be flagged instead of silently accepted.

11.2 Where did I choose simplicity over correctness, and what could break first?

I am initially choosing a document-specific parser instead of supporting every possible PDF format. This keeps the implementation manageable, but a substantially different layout or unusual table structure could cause extraction errors. I will document unsupported patterns and extend the parser when evidence from the supplied manual requires it.

11.3 What input did I not handle, and what happens when it appears?

An initial version may not reliably resolve a section that has been substantially rewritten and moved while retaining a similar heading. This could produce an incorrect version match. The system should mark the relationship as ambiguous and expose the limitation rather than claim the node is unchanged.

12. Limitations and Future Improvements

The initial implementation targets the supplied CT-200 manuals and their observed structural variations; it is not a universal PDF parser. OCR errors, complex multi-column layouts, ambiguous section matching and semantic changes that do not alter much of the text remain potential limitations. LLM-generated test cases may also be incomplete or incorrect and require human review.

With additional development time, I would improve layout-aware table reconstruction, add regression tests for difficult pages, strengthen semantic change detection and introduce review workflows for ambiguous node matches.

13. Conclusion

The proposed system combines document extraction, hierarchical storage, version comparison, traceable selections and LLM-based QA test-case generation in one backend workflow. Its central design principle is to preserve the relationship between original document content and every derived result. By retaining historical versions, validating extraction and model output, and explicitly reporting uncertainty, the system aims to make document changes and generated QA ideas easier to inspect and verify.

Submission reminder: Before submitting, replace all placeholders with actual findings, implementation details and test results. Ensure the document accurately reflects the code in the GitHub repository.